Scenario Set Question 2

Scenario1: *You are developing a banking application that categorizes transactions based on the amount entered.*
Write logic to determine whether the amount is positive, negative, or zero.

Solution:

Step 1: Ask the user to enter the amount.

Step 2: Verify the amount using an if conditional statement to check if the amount is greater than 0 or not. If 'YES → print it as "positive amount"; 'NO' →proceed to step 3

Step 3: Verify the amount using an if conditional statement to check if the amount is less than 0 or not. If 'YES' → print it as "negative amount" ; 'NO' →proceed to step 4

Step 4: Verify the amount using an if conditional statement to check if the amount is equal to 0 or not. If 'YES' → print it as "zero amount" and return the value.

Scenario2: *A digital locker requires users to enter a numerical passcode. As part of a security feature, the system checks the sum of the digits of the passcode.*
Write logic to compute the sum of the digits of a given number.

Solution:

Step 1: Ask the user to enter the passcode and store it as x

Step 2: Read the sum of default set passcode value as y

Step 3: Covert the x as individual digits using "typecast"

Step 4: Create an empty variable z and append the empty variable by adding sum of the typecasted x value

Step 5: Return z

==========

Scenario3: *A mobile payment app uses a simple checksum validation where reversing a transaction ID helps detect fraud.*
Write logic to take a number and return its reverse.

Solution:

Step 1: Ask the user to enter the transaction id and store it as x

Step 2: Read the x and convert the string into a list of characters.

Step 3: Create an empty string as y, to store the reversed result, and Loop through each character in the original list.

Step 4: Prepend each character to the result string from the list and print y

---

Scenario4: *In a secure login system, certain features are enabled only for users with prime-numbered user IDs.*
Write logic to check if a given number is prime.

Solution:

Step 1: Read the number n

Step 2: Read the x and convert the string into a list of characters.

Step 3: If n <= 1, print "Not Prime" and Stop

Step 4: Set i = 2

Step 5: If i >= n, go to Step 8

Step 6: If n % i == 0, print "Not Prime" and Stop

Step 7: i = i + 1, go to Step 5

Step 8: Print "Prime"

Step 9: Stop

==========

Scenario5: *A scientist is working on permutations and needs to calculate the factorial of numbers frequently.*
Write logic to find the factorial of a given number using recursion.

Solution:

Step 1: Get the input number from the user.

Step 2: If the number is negative, display an error message: "Factorial not defined for negative numbers."

Step 3: If the number is 0 or 1, the factorial is 1.

Step 4: Otherwise, multiply the number by the factorial of (number - 1).

Step 5: Finally, output the result.

Scenario6: *A unique lottery system assigns ticket numbers where only Armstrong numbers win the jackpot.*
Write logic to check whether a given number is an Armstrong number.

Solution:

Step 1: Accept the input number.

Step 2: Determine how many digits the number contains.

Step 3: Set a variable to hold the cumulative sum, starting at 0.

Step 4: Go through each digit of the number individually:

- Compute the digit raised to the power of the total digit count.
- Add that computed value to the running sum.

Step 5: After processing all digits, compare the final sum with the original number.

- If they match, display "Armstrong Number".
- Otherwise, display "Not an Armstrong Number".

==========

Scenario7: *A password manager needs to strengthen weak passwords by swapping the first and last characters of user-generated passwords.*
Write logic to perform this operation on a given string.

Solution:

Step 1: Read the input string as x

Step 2: Check if x as atleast 2 values using if statements

If 'NO' -> Print "Swapping not possible"

If 'YES' -> Proceed to step 3

Step 3: Convert the x into a integer and store as y

Step 4: From y, trim the first letter and store in a temp variable as a and typecast the remaining letters in y

Step 5: From y, trim the first letter and store in a temp variable as b typecast the remaining letters in y

Step 5: Now, concatenate the string in order -> b+y+a and print

---

Scenario8: *A low-level networking application requires decimal numbers to be converted into binary format before transmission.*
Write logic to convert a given decimal number into its binary equivalent.

Solution:

Step 1: Read the input decimal number.

Step 2: Initialize an empty string or list to store binary digits.

Step 3: Repeat until the number becomes 0:

Divide the number by 2.

Record the remainder (which will be either 0 or 1).

Update the number to the quotient (integer division).

Step 4: Reverse the order of recorded remainders (since the last remainder is the most significant bit).

Step 5: Output the reversed sequence as the binary equivalent.

Step 6: Special case: If the input is 0, the binary equivalent is "0".

Scenario9: *A text-processing tool helps summarize articles by identifying the most significant words.*
Write logic to find the longest word in a sentence.

Solution:

Step 1: Read the input sentence as a string.

Step 2: Trim any leading or trailing whitespace (optional but good practice).

Step 3: Split the sentence into individual words using spaces as delimiters.

- Initialize variables:
  - longest_word = "" (to store the longest word found)
  - max_length = 0 (to track the maximum length)

Step 4: Loop through each word in the split list:

Step 5: Remove any punctuation attached to the word (optional, for cleaner results).

Step 6: Calculate the length of the current word.

Step 7: If the current word's length is greater than max_length:

Step 8: Update max_length to the current word's length.

Step 9: Update longest_word to the current word.

Step 10: Optional: If there's a tie (equal length), decide whether to keep the first occurrence or the last.

Step 11: Output longest_word as the most significant (longest) word.

Scenario: *A plagiarism detection tool compares words from different documents and checks if they are anagrams (same characters but different order).*
Write logic to check whether two given strings are anagrams.

Solution:

Step 1: Read the two given strings

Step 2: Clean both strings (remove spaces, convert to lowercase).

Step 3: Sort the characters in both strings.

Step 4: Compare the sorted strings.

Step 5: If they match → Anagrams, else → Not Anagrams.
